De Bruijn-Style Higher-Order Workflow Memory for Agentic Systems

Research Notes

A de Bruijn-style higher-order workflow memory is a structured memory substrate for agents that represents *recent history* as an explicit discrete state (a length-k window of typed events) and represents *what tends to happen next* as outgoing edges annotated with empirical statistics and outcome metadata. This turns "memory retrieval" from semantic similarity search over past text into neighborhood lookup in a learned, inspectable transition graph, closely related to higher-order Markov models and higher-order networks.

Executive Summary

The key rigorous advantages (under explicit assumptions stated below) are:

1. **Retrieval efficiency and bounded candidate set size.** Given the current k-window state, next-step candidates are the out-neighbors of that state. Candidate set size is the state's outdegree $D(s)$, which can be far smaller than a fixed top-R chunk retrieval budget used by vector databases or RAG memory, and independent of total memory size N.[1]

2. **Lower error floors when dependencies are higher-order.** If the environment truly depends on history beyond order 1, any first-order model (or any memory mechanism that effectively collapses distinct k-contexts) incurs irreducible bias, while a higher-order representation can achieve Bayes-optimal next-step prediction in principle.[2]

3. **Compression and inspectability through repeated suffix merging.** The number of unique k-window states is at most $\min(T, |\Sigma|^k)$ for a trace length T but is often far smaller in real workflows with repeated phases and sparse branching.[3]

4. **Alignment with emerging agent memory research.** Recent graph-memory work argues that flat similarity retrieval entangles relation types and harms interpretability; multi-graph agent memory (semantic, temporal, causal, entity) is a response.[4]

The main formal limits are equally sharp: the worst-case state space grows as $|\Sigma|^k$ (state explosion), higher-order adjacency captures *predictive dependence* but not necessarily *causality*, and segmentation/-typing noise can smear the transition kernel.[5]

[1] This yields provable bounds on action candidate enumeration and prompt tokenization cost when the workflow dynamics are sparse. See the ANN and HNSW literature for sublinear query complexity claims.

[2] This is the same reason higher-order network models can reveal structure missed by first-order representations—a well-studied phenomenon in network science.

[3] This provides measurable compression against storing raw episodic text and enables human inspection of "what usually follows what" with counts, error rates, and time distributions.

[4] A de Bruijn-style layer cleanly instantiates a *higher-order temporal graph* that can be composed with systems like MAGMA (multi-graph traversal) and ActMem (causal and semantic memory graphs).

[5] These motivate variable-order back-off and Bayesian mixtures such as context-tree weighting (CTW), which provide principled adaptation when the effective memory order varies.

Formal Definitions

Typed Events and Workflow Traces

Let $\mathcal{E}$ be the space of raw observations generated by an agent's interaction loop.[6]

A **typed event** is a tuple

$$x_t = \big(\tau_t,\ r_t,\ o_t,\ a_t\big)$$

where:

- $\tau_t \in \mathcal{T}$: event type (e.g., `logs_requested`, `invoice_approved`),
- $r_t \in \mathcal{R}$: role or actor class (user, agent, service),
- $o_t \in \mathcal{O}$: object/entity class (ticket, document, patient-case),
- $a_t \in \mathcal{A}$: attribute payload (structured fields, status codes, tags).

A **typing function** ϕ maps raw observation $e_t \in \mathcal{E}$ to a discrete symbol in a finite alphabet Σ:

$$\sigma_t = \phi(x_t) \in \Sigma, \quad |\Sigma| = A.$$

A **workflow trace** of length T is a sequence $\sigma_{1:T} = (\sigma_1, \ldots, \sigma_T)$.

[6] Examples include tool calls, user messages, system actions, external events, and internal decisions.

The typing function may be hand-designed (schema rules), learned (event classifier), or hybrid.

De Bruijn-Style Higher-Order Workflow Memory

Fix an order $k \geq 1$. Define the **k-window state** at time t (for $t \geq k$) as:

$$s_t = (\sigma_{t-k+1}, \ldots, \sigma_t) \in \Sigma^k.$$

Define the **k-order de Bruijn-style workflow graph** $G_k = (V_k, E_k)$ where:[7]

- $V_k \subseteq \Sigma^k$ is the set of k-windows observed,
- $E_k \subseteq V_k \times V_k$ contains a directed edge $s \to s'$ if s' can follow s by shifting left and appending one symbol:

$$s = (u_1, \ldots, u_k), \quad s' = (u_2, \ldots, u_k, v)$$

for some $v \in \Sigma$.

Each edge is annotated with **edge statistics**:

$$\text{stats}(s \to s') = \Big(c,\ \hat{p},\ \widehat{\Delta t},\ \widehat{\text{succ}},\ \widehat{\text{fail_tags}},\ \ldots\Big)$$

[7] This is the same overlap construction that defines classical de Bruijn graphs (overlap by $k-1$ symbols), adapted from characters or k-mers to typed workflow events.

Typical components:

$c(s \to s')$: transition count
$\hat{p}(s \to s')$: estimated transition probability (smoothed)
$\widehat{\Delta t}$: empirical time-to-next distribution summary
$\widehat{\text{succ}}$: success rate conditional on taking the transition
Additional metadata: cost, reward, risk score, anomaly score.

Baseline Memory and Modeling Families

Flat episodic chunk store (RAG-style memory). Memory consists of N items $\{m_i\}_{i=1}^{N}$. An embedding function $f : \text{text} \to \mathbb{R}^d$ maps each chunk to a vector. Retrieval for query q returns top-R items by nearest neighbor search.[8]

[8] This is the canonical retrieval-augmented generation pattern. ANN indices such as HNSW scale sublinearly in N and can admit near-logarithmic query complexity.

Simple k-order Markov models. A k-order Markov model assumes:

$$\Pr(\sigma_{t+1} \mid \sigma_{1:t}) = \Pr(\sigma_{t+1} \mid \sigma_{t-k+1:t}).$$

This is equivalent to a first-order Markov chain over Σ^k.[9]

[9] Viewing de Bruijn graphs as order-m Markov chains appears explicitly in recent de Bruijn process work.

Graph-memory variants for LLM agents. Recent agent memory work increasingly uses explicit graphs:[10] A de Bruijn-style workflow layer is best viewed as a *specific temporal component* within this graph-memory family: it formalizes higher-order temporal adjacency beyond first-order event graphs.

[10] **MemGPT**: tiered memory management (memory as OS abstraction).
GraphRAG: knowledge graph over corpora for improved retrieval.
MAGMA: separate semantic/temporal/causal/entity graphs with traversal-based retrieval.
ActMem: causal and semantic graph bridging retrieval and action.
ES-Mem: event segmentation and hierarchical memory.

Theoretical Analyses

Assumptions

Let $A = |\Sigma|$. Let the data-generating process over Σ satisfy:

A1 (*k-order dependence*). There exists $k^\star$ such that σ_{t+1} depends on history only through $\sigma_{t-k^\star+1:t}$.

A2 (*Stationarity and ergodicity*). The induced chain over $\Sigma^{k^\star}$ is stationary and ergodic with stationary distribution π.

A3 (*Sparsity*). For each state $s \in \Sigma^{k^\star}$, at most D next symbols have nonzero probability:

$$|\{a \in \Sigma : P(a \mid s) > 0\}| \leq D \ll A.$$

Assumptions A1–A3 represent the "best case" for de Bruijn-style memory: the real workflow is governed by recent history and transitions are sparse. A4–A5 quantify the main practical challenges: noisy events and rare states.

A4 (*Typing/segmentation noise*). Observed symbols are possibly corrupted: $\tilde{\sigma}_t = \sigma_t$ with probability $1 - \eta$, else mapped to an incorrect symbol.[11]

[11] A simplified model of segmentation and typing errors, motivated by the need to treat event extraction as a noisy preprocessing step.

A5 (*Coverage lower bound*). The stationary distribution satisfies $\pi_{\min} = \min_{s \in V} \pi(s) > 0$.

Retrieval Efficiency and Candidate Set Size

Let N be the episodic chunk count, R the top-R retrieval count, T events observed, $M_k = |V_k|$ the distinct k-windows, and $\deg^+(s)$ the outdegree of state s.

Proposition 1 (Bounded candidate enumeration)**.** *For de Bruijn-style memory with adjacency lists, candidate enumeration time is $O(\deg^+(s_t))$, independent of N and T.*

For vector DB retrieval, the pipeline requires at least $\Omega(R)$ to enumerate retrieved items (and typically $\Omega(R \cdot L)$ in token budget, where L is chunk length).

Proof. Adjacency retrieval is a single hash lookup for s_t plus iteration over its outgoing list of $\deg^+(s_t)$ edges. For vector retrieval, at least R items must be produced. ANN accelerates *which* items are picked but does not reduce the output cardinality R.[12] □

[12] Sublinear ANN query time is documented for HNSW and related indices, but output size remains fixed at R.

Strict advantage regime. If the workflow is sparse (A3) so that $\deg^+(s_t) \leq D \ll R$, then de Bruijn candidate enumeration is strictly smaller in both R-independent cost and prompt token footprint.

State Coverage and Compression

Proposition 2 (Worst-case and observed-state bounds)**.** *For a trace of length T, the number of distinct observed k-window states satisfies:*

$$M_k \leq \min(T - k + 1,\ A^k).$$

Proposition 3 (Compression factor vs. raw episodic storage)**.** *If a flat store keeps all T events explicitly, whereas de Bruijn-style memory stores unique k-windows and their transitions, then the compression ratio is:*[13]

$$\mathrm{CR}_k = \frac{T - k + 1}{M_k}.$$

[13] When workflows repeat phases, M_k grows sublinearly in T and CR_k grows with T, yielding increasing compression. This is the same structural reason de Bruijn representations compress repeated overlaps in genomic sequence assembly.

Proposition 4 (Sparsity reduces edge storage)**.** *Under A3, total stored edges satisfy $|E_k| \leq D \cdot M_k$.*

Worst-case $|E_k|$ can be $A \cdot A^k = A^{k+1}$, but sparse workflows are closer to $D \cdot M_k$.

Next-Step Prediction Accuracy

Define the true conditional distribution $P(\cdot \mid s)$ for each k-state s. Let $\hat{P}_n(\cdot \mid s)$ be the empirical estimate from n_s occurrences. We measure error via expected total variation distance:

$$\mathrm{Err} = \mathbb{E}_{s \sim \pi}\left[\tfrac{1}{2}\|\hat{P}(\cdot \mid s) - P(\cdot \mid s)\|_1\right].$$

Theorem 1 (Consistency of de Bruijn transition estimation)**.** *Under A1–A2 and for any fixed s with $\pi(s) > 0$, $n_s \to \infty$ a.s. as $T \to \infty$, and $\hat{P}(\cdot \mid s) \to P(\cdot \mid s)$ almost surely.*

Proof sketch. In an ergodic chain, $n_s/T \to \pi(s) > 0$, hence $n_s \to \infty$. By the strong law for multinomial counts, empirical frequencies converge to the true transition probabilities.[14] □

[14] PAC-style estimation bounds for Markov kernels provide modern minimax references for finite-state transition estimation.

Theorem 2 (Higher-order strict dominance)**.** *Assume the true process is order $k^\star = 2$ with $\Sigma = \{0, 1\}$ and dynamics $\sigma_{t+1} = \sigma_t \oplus \sigma_{t-1}$ (XOR), with (σ_{t-1}, σ_t) uniformly distributed. Then:*[15]

- *The Bayes-optimal 2-order predictor achieves zero next-step error.*
- *Any 1-order predictor has error at least $1/2$ under 0–1 loss.*

[15] This matches the empirical and theoretical motivation for higher-order network models: first-order representations can lose crucial path information and mischaracterize dynamics.

Proof. Given (σ_{t-1}, σ_t), σ_{t+1} is deterministic, so error is 0. Given only σ_t, σ_{t-1} is equally likely to be 0 or 1 (by uniformity), so σ_{t+1} is equally likely to be 0 or 1; any predictor achieves correctness probability $1/2$. □

Sample Complexity to Reach ε-Error

Theorem 3 (Uniform TV estimation bound for sparse transitions)**.** *Fix k and let V_k be the set of states to estimate. Assume each state has support size at most D (A3) and receives at least $n_{\min}$ visits. Then for any $\delta \in (0, 1)$, if*

$$n_{\min} \geq \frac{2}{\varepsilon^2} \log\Big(\frac{2|V_k|D}{\delta}\Big),$$

we have with probability at least $1 - \delta$:

$$\max_{s \in V_k} \|\hat{P}(\cdot \mid s) - P(\cdot \mid s)\|_1 \leq \varepsilon.$$

Proof sketch. Apply Hoeffding's inequality to each symbol probability per state and union bound over D symbols and $|V_k|$ states.[16] □

[16] The key step translates per-coordinate deviation into an L_1 bound over the at-most-D supported symbols.

Corollary 1 (Total sample requirement under $\pi_{\min}$)**.** *If the chain visits each $s \in V_k$ with probability at least $\pi_{\min}$, sufficient total trace length is:*

$$T \gtrsim \frac{1}{\pi_{\min}} \cdot \frac{2}{\varepsilon^2} \log\Big(\frac{2|V_k|D}{\delta}\Big).$$

Scaling insight. Worst case $V_k = \Sigma^k$ gives $|V_k| = A^k$, implying $T = \tilde{O}(A^k D / \pi_{\min} \varepsilon^2)$—exponential in k. This is the formal state-explosion limit and the reason variable-order models (CTW, PST) are important: they adaptively allocate complexity where evidence supports it.

Retrieval Mismatch in Vector-Memory Baselines

Let s_t be the true state. Suppose vector retrieval produces surrogate $\tilde{s}_t = s_t$ with probability $1 - \alpha$ and is incorrect otherwise. Let $\Delta = \max_{s \neq s'} \|P(\cdot \mid s) - P(\cdot \mid s')\|_{\mathrm{TV}}$.

Proposition 5 (Error induced by retrieval mismatch)**.**

$$\mathbb{E}\big[\|P(\cdot \mid \tilde{s}_t) - P(\cdot \mid s_t)\|_{\mathrm{TV}}\big] \leq \alpha\Delta,$$

and can be as large as α when $\Delta = 1$ (disjoint supports).

When higher-order contexts produce sharply different next-step distributions (large Δ), approximate retrieval mismatch α directly translates into irreducible prediction error. De Bruijn adjacency retrieval sets $\alpha = 0$ when state identification is exact. This motivates graph-based and event-segmentation approaches over flat similarity-only retrieval.

Algorithms and Data Models

Data Model

A practical minimal schema (relational or key-value) for order k:

This is directly analogous to weighted de Bruijn graphs (edges weighted by k-mer counts) in sequence settings, but the payload is richer due to workflow metadata.

Node table.

- `state_id`: hash of k-tuple $(\sigma_{t-k+1}, \ldots, \sigma_t)$
- `k`: order
- `support_count`: number of times the state occurred
- `last_seen_ts`
- Optional: canonical decoded event tuples for readability

Edge table.

- `(state_id, next_symbol)` or `(state_id, next_state_id)`
- `count`, `p_hat` (optional materialized)
- `time_delta_stats` (mean, p50, p90)
- `success_rate` (if labeled)
- `risk_tags` or anomaly score

Construction and Update

Online update pseudocode, assuming the event stream provides σ_t:[17]

[17] This implements the overlap transition rule of higher-order de Bruijn graphs: shift left, append symbol.

```
procedure UPDATE_MEMORY(k, window_queue, G_k,
                  sigma_t, timestamp_t, outcome_t)
  push sigma_t into window_queue
  if length(window_queue) < k:
     return

  s = tuple(window_queue[-k : ])
  increment NodeCount[s]
  update NodeLastSeen[s] = timestamp_t

  if previous state s_prev exists:
     increment EdgeCount[s_prev, s]
     dt = timestamp_t - timestamp_prev
     update EdgeTimeStats[s_prev, s] with dt
     update EdgeOutcomeStats[s_prev, s] with outcome_t

  timestamp_prev = timestamp_t
  s_prev = s
```

Variable-Order Backoff

Because k can be too specific or sparse, implement backoff:[18]

```
procedure GET_CANDIDATES(max_k, context,
                   G_list, min_support)
  for k from max_k down to 1:
     s = suffix_k(context)
     if NodeCount[s] >= min_support:
        return OutNeighbors(s)
  return GLOBAL_PRIOR_CANDIDATES()
```

[18] This aligns with variable-order Markov approaches (probabilistic suffix trees, CTW) designed to handle heterogeneous effective memory order.

Belief Under Segmentation Uncertainty

If the system may mis-segment or have missing events, maintain a belief distribution over candidate states $b_t(s)$ and retrieve from a mixture:[19]

$$\hat{P}(\sigma_{t+1}) = \sum_s b_t(s)\, \hat{P}(\sigma_{t+1} \mid s).$$

[19] This is a standard hidden-state filtering pattern; the hidden state is the true k-window.

Experimental Design and Validation

Tasks and Metrics

Primary task: next-step prediction. Given recent history, predict σ_{t+1}. Metrics: top-1 accuracy, top-k accuracy, negative log-likelihood (cross-entropy), calibration (Brier score, ECE), per-state error stratified by support n_s.

Agent utility task: decision support. Use memory to propose next action candidates for an agent policy.[20]

Memory quality task. Map interactions into incremental multi-turn memory settings using benchmarks such as MemoryAgentBench.[21]

[20] Evaluate: completion rate on multi-step workflows, recovery rate after failures, token budget and latency per decision, human override rate in assisted settings.

[21] MemoryAgentBench targets accurate retrieval, test-time learning, long-range understanding, and related competencies.

Synthetic Datasets

Synthetic datasets allow provable properties ($k^\star$, D, η) to be manipulated directly.

S1: Sparse k-order Markov workflows. $A \in \{20, 50, 100, 200\}$, $D \in \{2, 5, 10\}$, $k^\star \in \{1, \ldots, 5\}$.

S2: Higher-order parity/XOR workflows (generalizing Theorem 2).

S3: Segmentation noise injection, $\eta \in \{0, 0.01, 0.05, 0.10\}$.

S4: Nonstationary drift with known change points.

Real Datasets

Public process mining event logs are a natural fit:[22]

- BPI Challenge logs (loan applications, problem management, purchase orders)
- Sepsis Cases event log (healthcare pathway traces)

[22] These logs enable evaluation of next activity prediction, time-to-next prediction, bottleneck detection, conformance, and drift.

- Additional XES logs from process mining repositories

Baselines

1. Flat chunk retrieval (RAG memory) with ANN vector search and fixed R
2. First-order Markov / event graph predictor
3. Fixed k-order Markov (without rich edge stats)
4. Variable-order Markov / CTW / PST predictors[23]
5. Graph-memory agent baselines (MAGMA, ActMem) where feasible

[23] Principled adaptive baselines with strong theoretical guarantees in prediction and compression.

Parameter Sweeps

Recommended sweeps: $k \in \{1, \ldots, 8\}$, $A \in \{20, 50, 100, 200\}$, $D \in \{2, 5, 10\}$, T on a logarithmic sweep, $\eta \in \{0, 0.01, 0.05, 0.10\}$, and backoff threshold `min_support`.

Suggested charts: NLL vs. T (log scale), TV error vs. T by k and D, candidate set size distribution $\deg^+(s)$ vs. fixed R, memory footprint vs. k, ablation of exact-k vs. variable-order vs. belief mixture.

Statistical Tests

For next-step prediction on sequence data, prefer paired tests:[24]

- Paired bootstrap over cases for NLL and accuracy
- Wilcoxon signed-rank test on per-case NLL differences
- Holm correction for multiple comparisons across k values

[24] Report effect sizes (mean NLL delta) and confidence intervals.

Failure Modes and Open Problems

State Explosion

Worst-case $M_k \approx A^k$ implies exponential growth with k, and Theorem 3 shows sample requirements can inherit that scaling.

Mitigations:
Variable-order backoff and Bayesian mixtures (CTW).
Sparsity-aware estimation (support truncation).
Hierarchical typing (coarser symbols for long-range, finer locally).

Non-Markovian Causality and Confounding

Higher-order adjacency captures predictive dependence, but "$s \to s'$" is not causal in general.[25]

A rigorous stance: de Bruijn-style memory provides a strongly specified *predictive* substrate; separate causal inference modules are needed for intervention claims.

[25] The representation preserves path memory but not intervention semantics. Separate causal inference modules (as in ActMem's counterfactual reasoning) are needed for intervention claims.

Segmentation and Typing Errors

Under A4, observed transitions are noisy, causing inflated apparent alphabet size, spurious edges, and biased probability estimates.[26]

Formal direction: treat ϕ as a noisy channel and analyze identifiability of $P(\cdot \mid s)$ under symbol corruption.

[26] ES-Mem highlights that memory quality depends critically on event segmentation granularity and coherent event boundaries—this is a first-class modeling issue, not a minor implementation detail.

Overfitting to Frequent but Bad Workflows

De Bruijn-style edges record *what happens*, not *what should happen*. Without success metrics, the system can reinforce suboptimal routines.[27]

[27] Mitigations: annotate edges with outcomes and optimize for expected utility rather than frequency; enforce exploration constraints; compare with MAGMA's task-intent-aware traversal policies.

Integration with Modern Agent Memory Stacks

A nontrivial open problem is optimal composition between:[28]

- semantic retrieval (for facts and documentation),
- higher-order workflow memory (for procedural next steps),
- causal/entity graphs (for constraint satisfaction and conflict resolution).

[28] GraphRAG demonstrates that mixed local/global retrieval over graph structures can outperform flat RAG for certain question classes, suggesting hybrid retrieval is often superior to monolithic vector similarity search.